Report - Tomasulo Algorithm Implementation

Zarif Ahmed- 2281643

1. Introduction

The goal of this assignment is to let an existing simulator issue one to four instructions per cycle, in program order, while keeping the results correct.
We started with the original python code and made changes for multi-issue, register tracking, R0, and branches. We finished the remaining memory fixes, added support for more execution units, improved recovery after a wrong prediction and added tests. We kept the five stages: ISSUE, EX, MEM, WB, and COMMIT.

2. Implementation

Issuing instructions and tracking inputs-

The simulator issues up to four instructions per cycle. The default is one. It stops issuing when the reorder buffer (ROB) or a reservation station is full. The register alias table (RAT) tracks which instruction will produce each register value. Instructions issued in the same cycle see the updates from earlier instructions, so they wait for the right inputs. Results commit in program order, and R0 always stays zero.

Hardware settings-

We added support for more than one execution unit. The input file sets the number of units, waiting spaces, and cycles each operation takes. An integer unit finishes one operation before starting another. Each floating-point (FP) unit can start a new operation every cycle. The oldest ready instruction goes first. Loads and stores use separate address adders but share one memory port.

Load/store correctness

We fixed loads so they can get data from the most recent earlier store to the same address. Stores to other addresses no longer block a ready load by mistake. Stores change memory only at COMMIT. If a load runs before an earlier store address is known, we check its value before it commits. If the value is wrong, we restart that load and the instructions after it.

Branch prediction and recovery.
The simulator uses a one-bit predictor to guess whether a branch will be taken. An eight-entry branch target buffer (BTB) stores branch destinations. It uses the last three bits of the word address of the instruction to choose an entry and checks the address tag. If the guess is wrong, we remove the later instructions and restore the RAT. A wrong guess found in cycle n allows correct instructions to issue in cycle n+2.

3. Experiments

We ran eleven test programs at issue widths 1, 2, 3, and 4, for 44 runs. Five programs were already in the code folder. We added six to test memory forwarding, restarting incorrect loads, wrong branch paths, loops, independent instructions, and the benefit of wider issue. For each program, we kept the hardware settings the same and changed only the issue width. We wrote a separate, simple program that executes instructions one at a time to calculate the expected answers. We compared every register, memory value, and committed instruction count with those answers. We also ran 30 randomly generated programs at all four widths, giving 120 more runs. Fixed random seeds make these tests repeatable.

4. Results

The table shows the total cycles for each test program. All four issue widths give the same final register and memory values. All 44 runs also match the answers from the simple one-instruction-at-a-time program.

Benchmark	Committed	Width 1	Width 2	Width 3	Width 4
address_test	2	14	14	14	14
branch_test	8	15	15	15	15
code	25	58	58	58	58
r0_test	4	7	7	7	7
rename_test	3	13	13	13	13
branch_loop	24	63	63	63	63
forwarding	6	18	19	19	19
independent	8	18	18	18	18
issue_overlap	13	35	29	27	26
load_replay	4	27	27	27	27
wrong_path	3	18	19	18	18

The issue_overlap test has twelve independent integer instructions followed by a long FP multiply. A wider issue lets the multiply start sooner. Width 1 takes 35

cycles, while width 4 takes 26 cycles. This is a 1.35x speedup. At width 4, the first four instructions all issue in the first cycle.

The tables below show when each instruction passes through the stages in issue_overlap. Integer EX takes 1 cycle, the multiply takes 20 cycles, and the ROB holds 32 entries. Both widths finish with R1 through R12 equal to 1 through 12, and F3=6. We leave out the unused MEM column here, but the full output files includes it.

Issue width 1

Instruction	ISSUE	EX	WB	COMMIT
Addi R1 R0 1	[1]	[2, 2]	[3]	[4]
Addi R2 R0 2	[2]	[3, 3]	[4]	[5]
Addi R3 R0 3	[3]	[4, 4]	[5]	[6]
Addi R4 R0 4	[4]	[5, 5]	[6]	[7]
Addi R5 R0 5	[5]	[6, 6]	[7]	[8]
Addi R6 R0 6	[6]	[7, 7]	[8]	[9]
Addi R7 R0 7	[7]	[8, 8]	[9]	[10]
Addi R8 R0 8	[8]	[9, 9]	[10]	[11]
Addi R9 R0 9	[9]	[10, 10]	[11]	[12]
Addi R10 R0 10	[10]	[11, 11]	[12]	[13]
Addi R11 R0 11	[11]	[12, 12]	[13]	[14]
Addi R12 R0 12	[12]	[13, 13]	[14]	[15]
Mult.d F3 F1 F2	[13]	[14, 33]	[34]	[35]

Issue width 4

Instruction	ISSUE	EX	WB	COMMIT
Addi R1 R0 1	[1]	[2, 2]	[3]	[4]
Addi R2 R0 2	[1]	[3, 3]	[4]	[5]
Addi R3 R0 3	[1]	[4, 4]	[5]	[6]
Addi R4 R0 4	[1]	[5, 5]	[6]	[7]
Addi R5 R0 5	[2]	[6, 6]	[7]	[8]
Addi R6 R0 6	[2]	[7, 7]	[8]	[9]
Addi R7 R0 7	[2]	[8, 8]	[9]	[10]
Addi R8 R0 8	[2]	[9, 9]	[10]	[11]
Addi R9 R0 9	[3]	[10, 10]	[11]	[12]
Addi R10 R0 10	[3]	[11, 11]	[12]	[13]
Addi R11 R0 11	[3]	[12, 12]	[13]	[14]
Addi R12 R0 12	[3]	[13, 13]	[14]	[15]
Mult.d F3 F1 F2	[4]	[5, 24]	[25]	[26]